Universidad Tecnológica de Chihuahua

Tecnologías de la Información y Comunicación

Área: Desarrollo y Gestión de Software

10N Aplicaciones web Progresivas (Alfonzo Barroso)

Diseño preproyecto PWA

IDGS102N

Vitaliano Hugo Cordova Herrera

Daniel Eduardo Peregrino Lechuga

Sergio Ramos Ramos

Jesus Asbed Rodriguez Canales

Jesús Alexis Morales González

Juan Hilario Gonzalez Perez

Chihuahua, Chih., Mex.

Fecha de entrega el reporte: 10/10/2026

Índice

Diseño preproyecto PWA **1**

Resumen ejecutivo **3**

Correspondencia con los requisitos 4

Introducción **4**

Objetivos **5**

Objetivo general 5

Objetivos específicos 5

Alcance funcional **5**

Arquitectura propuesta **6**

Arquitectura interna del backend 6

Responsabilidades 7

Consumo de API **8**

Servicios principales 9

..... 9

Configuración de URLs 9

Estrategia PWA **9**

Componentes implementados 10

Experiencia de usuario **11**

Organización modular 11

Vistas principales 11

Criterios de usabilidad 11

Implementación y pruebas 12

Tecnologías 12

Ejecución local 12

Pruebas realizadas 12

Seguridad y mejoras futuras 13

Conclusiones 14

Repositorio Git 14

Resumen ejecutivo

Fix IT es una Aplicación Web Progresiva orientada al aprendizaje de diagnóstico y reparación de computadoras mediante una experiencia de juego. La solución integra una interfaz React instalable, tres servicios REST desarrollados con Node.js y Express, y una base de datos MongoDB. El resultado permite registrar usuarios, consultar perfiles, mostrar estadísticas, administrar progreso y presentar un leaderboard con datos recuperados dinámicamente.

La propuesta satisface los componentes solicitados: consumo de API para recuperar y renderizar información; manifiesto y Service Worker para instalación y soporte offline; y una interfaz modular y adaptativa. La arquitectura mantiene separadas las responsabilidades del frontend, los servicios y la persistencia, lo que facilita su mantenimiento y crecimiento.

Correspondencia con los requisitos

Componente	Implementación en Fix IT
Consumo de API	Servicios REST para usuarios, progreso y sesiones; intercambio de datos mediante HTTP y JSON.
Estrategia PWA	Manifest, Service Worker, Cache API, instalación en Chrome y Edge y apertura básica sin conexión.
UI y UX	React, componentes reutilizables, CSS modular, navegación protegida y diseño adaptativo.
Documentación	README, diagramas, instrucciones de ejecución y URL del repositorio Git.

Introducción

Las PWA combinan el alcance de una aplicación web con capacidades propias de una aplicación instalada. Fix IT adopta este enfoque para que el usuario pueda acceder desde el navegador, instalar la interfaz en el equipo y conservar recursos esenciales para abrirla cuando no exista conexión a internet.

El juego se inspira en experiencias cooperativas de administración de tareas, pero traslada la dinámica al diagnóstico y reparación de computadoras. Cada sesión puede producir puntuación, estrellas, tickets atendidos y tiempo de juego; estos datos alimentan las vistas de perfil, estadísticas y clasificación.

Objetivos

Objetivo general

Diseñar e integrar la arquitectura y la interfaz de una PWA para Fix IT, conectando una experiencia de usuario responsiva con servicios REST y una base de datos persistente, e incorporando instalación y soporte offline básico.

Objetivos específicos

- Consumir endpoints remotos y representar los resultados dinámicamente en la interfaz.
- Separar usuarios, progreso y sesiones en servicios con responsabilidades definidas.
- Configurar un manifiesto y un Service Worker para instalación, caché y apertura sin conexión.
- Organizar la interfaz mediante componentes, páginas, contexto de autenticación y estilos adaptativos.
- Documentar la solución con diagramas, instrucciones y evidencias reproducibles.

Alcance funcional

- Registro e inicio de sesión de usuarios.
- Página de inicio con presentación del juego y acceso a misiones.
- Perfil con nivel, puntuación, tickets y tiempo jugado.
- Estadísticas y resumen de reparaciones.
- Leaderboard de jugadores.
- Registro y consulta de progreso y sesiones.
- Instalación desde Chrome o Edge y uso básico del App Shell sin internet.

Arquitectura propuesta

La solución sigue una arquitectura cliente servidor con separación por servicios. El frontend concentra la navegación y la presentación; el backend expone operaciones REST; MongoDB conserva los documentos de usuarios, progreso y sesiones; y el Service Worker administra los recursos estáticos necesarios para la experiencia PWA.

Arquitectura interna del backend

El backend de Fix-It_Int utiliza una arquitectura basada en tres microservicios independientes, organizados mediante separación de responsabilidades y comunicación REST.

Cada microservicio sigue una estructura por capas:

Route → Controller → Service → Repository → MongoDB

Route: define los endpoints y métodos HTTP disponibles.

Controller: recibe solicitudes HTTP y construye las respuestas correspondientes.

Service: implementa las reglas de negocio y validaciones.

Repository: ejecuta operaciones de consulta y persistencia.

Model: define los esquemas de datos mediante Mongoose.

Validator: comprueba la validez de los datos recibidos.

Esta organización facilita el mantenimiento, las pruebas y la evolución independiente de cada servicio.



Figura 1. Arquitectura general de Fix IT.

Responsabilidades

Capa	Responsabilidad
Frontend React	Presentar vistas, proteger rutas, capturar acciones y consumir las APIs.
User Service	Administrar usuarios, acceso, estadísticas generales y leaderboard.
Progress Service	Consultar y actualizar el avance de cada jugador.
Session Service	Registrar partidas y consolidar resultados de juego.
MongoDB	Persistir los datos usados por los servicios.
Service Worker	Interceptar solicitudes de recursos y atender el App Shell desde caché.

Consumo de API

El frontend recupera la información por medio de solicitudes HTTP y recibe respuestas JSON. Al iniciar sesión o abrir una vista, React solicita los datos correspondientes, actualiza el estado de la aplicación y renderiza listas, tarjetas y métricas sin recargar toda la página.

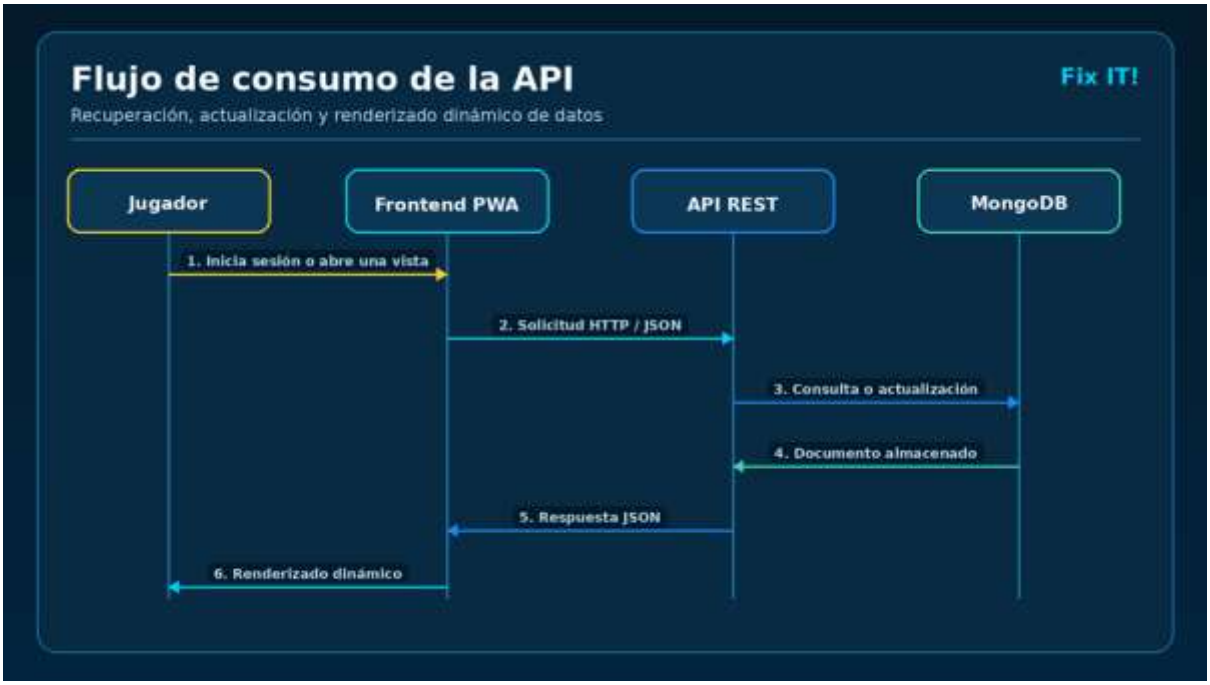


Figura 2. Secuencia de recuperación y renderizado de información.

Servicios principales



Figura 3. Servicios REST utilizados por el frontend.

Configuración de URLs

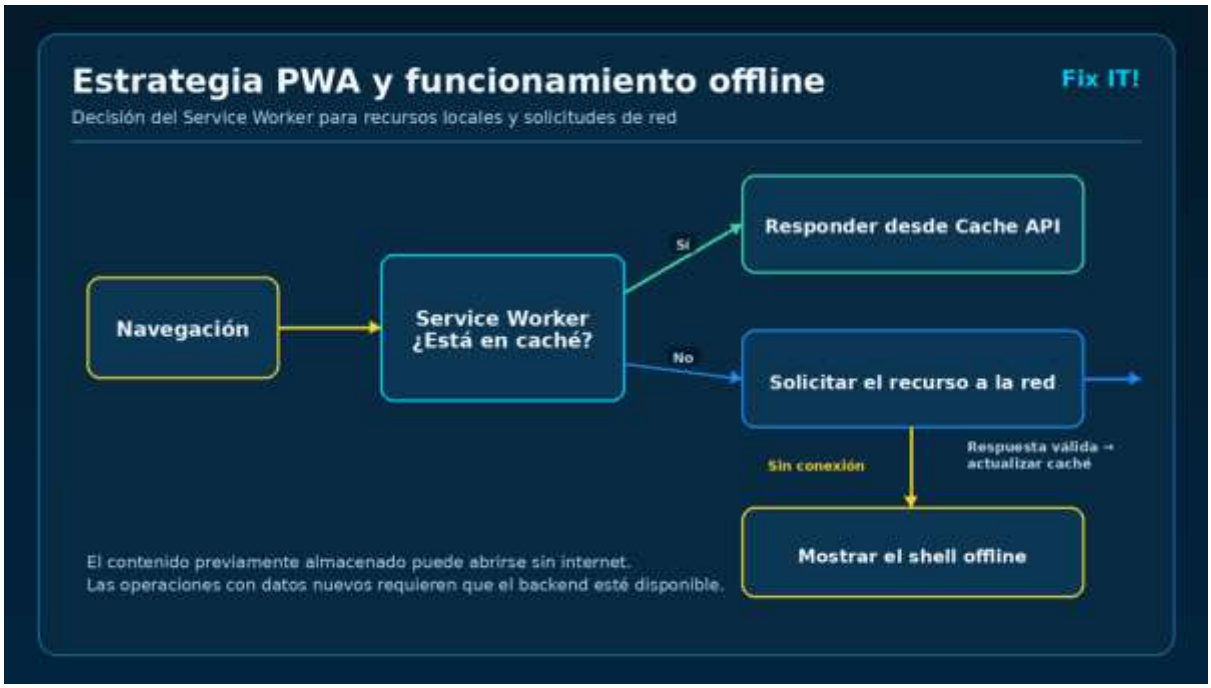
Las direcciones se mantienen fuera del código de las vistas mediante variables de entorno:

Identificador en .env	Valor
VITE_USER_API_URL	http://localhost:3001
VITE_PROGRESS_API_URL	http://localhost:3002
VITE_SESSION_API_UR	http://localhost:3003

Estrategia PWA

La estrategia PWA se compone del manifiesto, el registro del Service Worker y una política de caché. El manifiesto define el nombre, los iconos, los colores, la URL inicial y el modo standalone. El Service Worker se ejecuta separado del documento abierto, intercepta solicitudes y permite responder con recursos almacenados.

Figura 4. Decisión de caché y operación sin conexión.



Componentes implementados

Elemento	Función
manifest.json	Describe la aplicación instalable, sus iconos, colores, inicio y forma de visualización.
sw.js	Administra instalación, activación, caché y respuesta a solicitudes de recursos.
main.jsx	Registra el Service Worker al cargar la aplicación.
InstallPWA.jsx	Captura el evento de instalación y ofrece el botón Instalar aplicación.
Cache API	Guarda el App Shell, estilos, imágenes e iconos necesarios.

El alcance offline actual cubre la apertura de la interfaz y los recursos previamente almacenados. Las operaciones con datos nuevos necesitan que MongoDB y los servicios del backend estén disponibles. La sincronización diferida de partidas mediante Background Sync se considera una mejora futura.

Experiencia de usuario

La interfaz utiliza una identidad visual tecnológica basada en tonos azul oscuro y cian, tarjetas con jerarquía clara y navegación consistente. Los componentes permiten reutilizar encabezados, menús, botones de instalación y elementos de datos. El diseño adaptativo reorganiza el contenido según el ancho disponible.

Organización modular

Módulo	Contenido
components	Elementos reutilizables, navegación y control de instalación PWA.
pages	Login, Inicio, Estadísticas, Leaderboard y Perfil.
context	Estado de autenticación compartido y protección de rutas.
services	Funciones responsables de comunicarse con las APIs.
styles	Reglas visuales, adaptación responsiva, animaciones y estados interactivos.

Vistas principales

- **Acceso y registro:** Entrada del usuario y creación de cuenta.
- **Inicio:** Descripción del juego, llamada a comenzar una misión y footer social.
- **Estadísticas:** Indicadores, progreso semanal y reparaciones recientes.
- **Leaderboard:** Clasificación de jugadores y sus resultados.
- **Perfil:** Nivel, progreso, resumen y logros del usuario.

Criterios de usabilidad

- Navegación visible y consistente entre secciones.
- Mensajes y acciones principales escritos en español.
- Contraste suficiente entre fondo, texto y controles.

- Retroalimentación en inicio de sesión, registro e instalación.
- Rutas privadas que redirigen al acceso cuando no existe usuario autenticado.

Implementación y pruebas

Tecnologías

Área	Tecnologías
Frontend	React 19, Vite, React Router, JavaScript, JSX y CSS.
PWA	Web App Manifest, Service Worker y Cache API.
Backend	Node.js, Express y Mongoose.
Datos	MongoDB y MongoDB Compass.
Control de versiones	Git y GitHub.

Ejecución local

1. Iniciar el servicio de MongoDB.
2. Ejecutar User Service en el puerto 3001.
3. Ejecutar Progress Service en el puerto 3002.
4. Ejecutar Session Service en el puerto 3003.
5. Ejecutar el frontend Vite en http://localhost:5173.

Pruebas realizadas

- Registro de un nuevo usuario y confirmación del documento en MongoDB Compass.
- Inicio de sesión y navegación entre las rutas protegidas.
- Consulta de información desde los tres servicios locales.
- Instalación de la PWA desde Chrome o Edge.
- Apertura de la interfaz con la conexión a internet desactivada.

- Cierre de sesión y redirección al acceso.

Seguridad y mejoras futuras

Los archivos .env y las dependencias quedan excluidos mediante .gitignore, mientras que .env.example documenta la configuración sin almacenar secretos. Para una publicación real se recomienda incorporar JWT, expiración de sesiones, validación estricta, rate limiting, HTTPS y una política CORS limitada al dominio de producción.

- Conectar completamente el botón Comenzar misión con el juego.
- Enviar puntuación, tickets, estrellas y tiempo al terminar una partida.
- Incorporar recuperación de contraseña y edición de perfil o avatar.
- Sincronizar acciones offline cuando la conexión vuelva a estar disponible.
- Desplegar frontend, servicios y base de datos en infraestructura remota.

Conclusiones

La propuesta integra los elementos esenciales de una PWA dentro de una arquitectura clara y comprobable. El frontend consume datos dinámicos, los servicios separan las responsabilidades y MongoDB conserva la información del jugador. El manifiesto y el Service Worker permiten instalar la aplicación y mantener disponible su interfaz principal sin conexión.

La organización modular favorece el mantenimiento del proyecto y permite reemplazar imágenes, ampliar el juego o desplegar los servicios sin rediseñar toda la solución. El repositorio integrado reúne frontend, backend, documentación y diagramas, lo que facilita su revisión académica y reproducción local.

Repositorio Git

El repositorio definitivo contiene el código fuente del frontend y backend, los archivos PWA, el README con instrucciones y los diagramas de la aplicación.

https://github.com/JuanUtch123/-FIXIT-Proyecto-Integrador